

中国科学技术大学

高质量实时渲染 技术挑战赛



基于 HarmonyOS 的透明泡泡实时折射 与色散渲染系统

队伍名称： 吃福是亏

队伍成员： 杨弘宇、常毅寒、杨硕、高凡、赵艺博

指导老师： 蔡有城

University of Science and Technology of China
CCF CAD/CG 2026 High-Quality
Real-Time Rendering Technology
Challenge



**Real-Time Transparent Bubble
Rendering with Refraction and
Iridescence on HarmonyOS**

Team Members: Hongyu Yang, Yihan Chang, Shuo Yang, Fan Gao, Yibo Zhao

Supervisor: Youcheng Cai

摘 要

本项目面向【CCF CAD/CG 2026】高质量实时渲染技术挑战赛赛题二”基于鸿蒙的透明物体实时色散渲染系统”，实现了一个运行于 HarmonyOS 设备上的透明泡泡实时渲染与交互系统。系统采用 HarmonyOS ArkTS + XComponent + NAPI + C++ OpenGL ES 3.0 的整体架构。在初赛工作的基础上，决赛版本重点扩展了多泡泡交互系统：建立了基于空间哈希和接触图的多泡泡仿真框架，实现了接触圆动力学、持久表面控制点变形、共享膜渲染、Young-Laplace 弯曲共享膜、以及分离/稳定双泡/融合三种接触结果；引入了风场系统和每泡 DBSTT 涡流片仿真；支持运行时动态创建与删除泡泡。本文按照渲染管线、光学模型、单泡物理模拟、多泡泡交互系统、交互演示与结果分析的顺序，介绍决赛提交版本的系统设计、关键实现与当前局限。

关键词：肥皂泡渲染，HarmonyOS，屏幕空间折射，薄膜干涉，虹彩，多泡泡交互，接触图，共享膜，融合，OpenGL ES

ABSTRACT

This project targets Track 2, "Real-Time Dispersion Rendering of Transparent Objects on HarmonyOS," and implements a real-time transparent bubble rendering and interaction system on HarmonyOS devices. The system is built with HarmonyOS ArkTS, XComponent, NAPI, and C++ OpenGL ES 3.0. Building on the preliminary-round submission, the final-round version adds a comprehensive multi-bubble interaction system: a contact-graph-based simulation framework with spatial hashing, contact-circle dynamics, persistent surface control points, shared-film rendering, Young-Laplace curved shared films, and three contact outcomes (separation, stable double bubble, fusion). It also introduces a wind field system, per-bubble DBSTT vortex sheet simulation, and runtime bubble creation/deletion. This report presents the final-round submission covering the rendering pipeline, optical models, single-bubble physics, multi-bubble interaction, demonstration features, and current limitations.

Key Words: soap bubble rendering; HarmonyOS; screen-space refraction; thin-film interference; iridescence; multi-bubble interaction; contact graph; shared film; fusion; OpenGL ES

目 录

第 1 章 引言	1
1.1 赛题目标与总体结构	1
1.2 系统组成	2
1.3 运行环境	3
第 2 章 渲染管线设计	4
2.1 渲染管线	4
2.1.1 多阶段屏幕空间折射	4
2.1.2 FBO 尺寸策略与坐标修正	5
2.2 屏幕空间折射实现	5
2.2.1 折射方向	5
2.2.2 边缘增强与薄膜厚度影响	6
2.3 薄膜虹彩模型	7
2.3.1 薄膜干涉基础	7
2.3.2 Kim2012 模式：实时 RGB 波长采样	7
2.3.3 Belcour Airy 模式：多波长 Airy 反射近似	8
2.3.4 不同模式对比	8
2.4 动态膜厚与颜色合成	9
2.4.1 动态膜厚	9
2.4.2 颜色合成	9
第 3 章 物理模拟与动力学	10
3.1 涡流片模型的理论基础	10
3.2 离散化与数值实现	11
3.3 与本项目渲染管线的集成	14
第 4 章 多泡泡交互系统	16
4.1 系统概述	16
4.2 核心数据结构	16
4.2.1 泡泡对象：DisplayBubble	16
4.2.2 接触关系：BubbleContactPair	17
4.2.3 表面控制点：SurfaceControl	17
4.3 空间哈希与 Broad Phase	18
4.4 接触激活与连续过渡	18

4.5 接触圆动力学与位置约束	19
4.5.1 接触圆半径	19
4.5.2 多接触位置约束迭代	19
4.6 持久表面控制点系统	19
4.6.1 弹簧-阻尼更新	19
4.6.2 接触区域的变形目标	19
4.7 接触区域速度共享	20
4.8 共享膜渲染	20
4.9 弯曲共享膜与 Young-Laplace 压力	21
4.10 接触结果选择	22
4.10.1 分离	22
4.10.2 稳定 8 字双泡	23
4.10.3 融合	23
4.11 风场系统	24
4.12 每泡 DBSTT 涡流片仿真	24
4.13 泡泡破裂机制	25
4.13.1 渲染管线的多重防护	26
4.13.2 鸿蒙端异步破裂仿真	27
4.14 渲染连续性与体积守恒	27
4.14.1 三层连续性策略	27
4.14.2 体积守恒	27
4.15 动态泡泡管理	27
4.16 当前局限	28
第 5 章 交互与演示	29
5.1 交互实现	29
5.1.1 触控交互 (HarmonyOS 端)	29
5.1.2 桌面端开发环境	29
5.1.3 界面参数	29
5.2 演示场景	30
5.2.1 综合开场场景 (Scene 按钮)	30
5.2.2 双泡专项演示 (Demo 按钮)	30
5.3 风场演示	31

第 6 章 结果与分析	32
6.1 系统成果	32
6.1.1 渲染与光学	32
6.1.2 多泡泡交互	32
6.1.3 动力学与交互	32
6.1.4 工程完整性	33
6.2 效果展示	33
6.2.1 多泡泡接触场景	33
6.2.2 共享膜与接触圆	34
6.2.3 三种接触结果对比	34
6.2.4 弯曲共享膜	35
6.3 性能分析	35
6.4 局限与展望	35
6.5 总结	36
参考文献	37

第 1 章 引 言

1.1 赛题目标与总体结构

赛题二“基于鸿蒙的透明物体实时色散渲染系统”要求参赛者在给定基线工程上完成透明物体的实时折射、色散与交互展示，并进一步体现多泡泡效果、运动模拟、完整 UI 和稳定运行能力。肥皂泡恰好同时包含透明折射、薄膜干涉、动态表面和交互反馈四类典型问题，因此非常适合作为赛题落地对象。

本技术文档将介绍本队最终决赛提交版本的核心技术路线。决赛版本在初赛基础上，将系统从”单主泡泡 + 装饰展示泡泡”架构扩展为完整的多泡泡交互系统：泡泡之间可以动态接触、形成共享膜、融合吸收或分离，并支持运行时增删泡泡与方向性风场。实现上，系统被组织为渲染管线、薄膜材质、几何数据、动态模拟和资源加载几个相对独立的模块，并通过 ArkTS 页面层与 Native 渲染层共同完成最终应用。

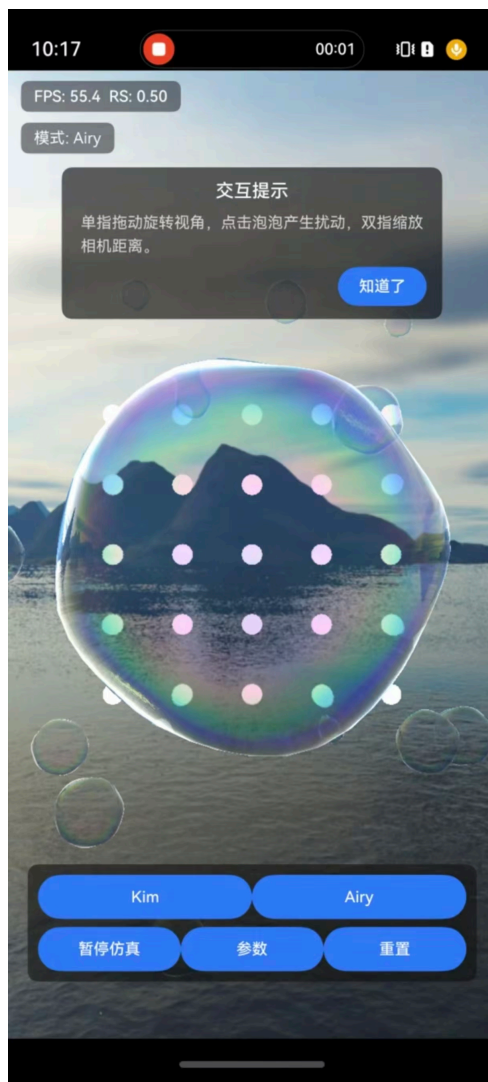


图 1.1 最终整体效果：多泡泡交互场景，包含背景折射、天空盒环境、虹彩边缘与泡泡间接触共享膜。

1.2 系统组成

最终鸿蒙提交版本的核心源码位于：

HarmonyOS_Source_Submission/refraction/

下表列出了和实现相关的各个模块。

表 1.1 系统模块与职责

模块	主要内容	作用
页面与交互	ArkTS 页面、XComponent、按钮与参数面板	承载渲染画面，处理触控输入、模式切换、状态显示和参数调节。
Native 接口与管线	NAPI、EGL、FBO、绘制顺序	连接 ArkTS 与 C++ 渲染层，组织多阶段折射渲染和场景合成。
泡泡材质	折射 shader、薄膜虹彩、动态膜厚	实现屏幕空间折射、Kim2012/LUT/Belcour Airy 三种虹彩模式和触摸波纹。
渲染封装	相机、模型、网格、shader 工具类	管理天空盒、球体、动态网格和 OpenGL ES 绘制细节。
表面模拟	DBSTT/vortex sheet 单泡泡动态	让主泡泡轮廓和法线随时间产生轻微形变。
多泡泡交互	接触图、空间哈希、表面控制点、共享膜	管理泡泡间接触检测、接触圆动力学、持久表面变形、共享膜生成与弯曲、融合/分离事件。
风场与动态管理	方向性风场、动态增删泡泡	提供可调节的全局风场，支持运行时按需创建和删除泡泡。
资源文件	天空盒、薄膜 LUT、HAP 提交资源	提供环境反射、背景折射参照、预计算薄膜反射数据和移动端运行素材。

1.3 运行环境

本项目初期开发在 Windows 主机上进行，验证设计可行性和效果后使用 DevEco Studio 完成基于 HarmonyOS 架构的实现。推荐环境如下：

表 1.2 推荐编译与运行环境

项目	要求
开发主机操作系统	Windows 11。
开发工具	DevEco Studio 6.0.2。
SDK 环境	HarmonyOS SDK 6.0.2 (API 22)。
页面层	ArkTS + XComponent。
Native 层	CMake + C++ + EGL + OpenGL ES 3.0 + NAPI。
真机测试	华为 Mate 70 Pro (HarmonyOS 6.0, API 22)。
模拟器	HarmonyOS 本地模拟器。

命令行构建可使用 Hvigor，例如：

```
node "E:\huawei\DevEco_Studio\tools\hvigor\bin\hvigorw.js"
assembleHap --no-daemon
```

签名完成后，构建产物通常位于：

```
entry/build/default/outputs/default/entry-default-signed.hap
```

提交的文件中已经整理出 HarmonyOS_Source_Submission 下的“可执行程序”目录。

第2章 渲染管线设计

2.1 渲染管线

2.1.1 多阶段屏幕空间折射

实时折射如果完全按路径追踪计算，需要跟踪光线穿过薄膜前表面、后表面以及环境的多次传播，开销较大。项目采用 Wyman 的 image-space refraction 思路^[1]：先把可被折射的背景渲染成纹理，再在泡泡 shader 中根据法线和视线方向扰动采样坐标，从而得到近似折射效果。为了在鸿蒙端同时组织主泡泡、展示泡泡和前后景关系，系统在 Native 层采用多阶段 FBO 合成，但核心光学思路仍然是用两个屏幕空间界面近似肥皂泡的双界面折射。

当前版本的三阶段屏幕空间折射流程

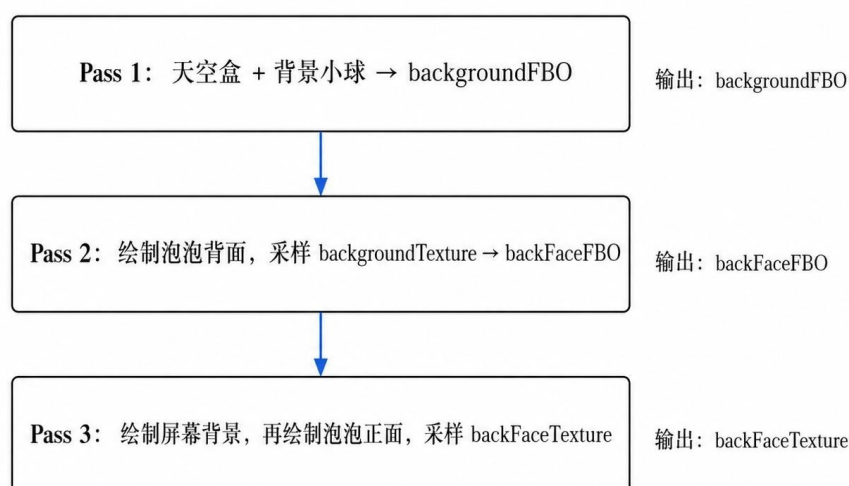


图 2.1 屏幕空间折射主线与多阶段 FBO 合成关系。

这一主线被组织为以下几个连续阶段：

1. **背景阶段**：将天空盒和背景参照物渲染到 backgroundFBO。
2. **后景阶段**：把主泡泡后方场景写入 sceneBehindFBO，为背面折射提供输入。
3. **背面阶段**：开启正面剔除，只绘制主泡泡背面，并输出到 backFaceFBO。
4. **主场景阶段**：绘制包含交互泡泡的场景到 mainSceneFBO，供后续合成采样。
5. **最终合成阶段**：将所有泡泡合成到 finalSceneFBO，再输出到屏幕。

这种方法的优点是完全运行在 GPU rasterization 管线中，不需要逐像素射线

求交；缺点是折射对象必须已经出现在屏幕空间纹理中，因此它是视觉上可信的近似，而不是严格物理光线追踪。对于鸿蒙终端场景而言，这种取舍能够在移动端预算下同时满足折射观感、多泡泡层次和交互响应速度。

2.1.2 FBO 尺寸策略与坐标修正

系统在 InitFBO 中采用低分辨率 FBO 合成，再由全屏四边形放大到最终屏幕，以兼顾移动端性能与画面完整性。其核心策略可写为：

$$W_{\text{fbo}} = 0.5W_{\text{screen}}, \quad H_{\text{fbo}} = 0.5H_{\text{screen}}.$$

由于 FBO 分辨率与窗口分辨率不同，正面渲染到屏幕时，gl_FragCoord 是窗口像素坐标；背面渲染到 FBO 时，gl_FragCoord 是 FBO 像素坐标。项目通过 uRenderToFBO 区分两种情况：

```
vec2 fboPixel = (uRenderToFBO == 1)
    ? screenPixel
    : screenPixel * (uFBOSize / uWinResolution);
```

该修正对多泡泡渲染尤其重要。各泡泡在不同 pass 中复用同一套折射与虹彩 shader，如果输入坐标不能正确映射到 FBO 空间，就会出现采样越界或局部发黑。修正后，所有交互泡泡都沿用统一材质逻辑，并通过模型矩阵、局部几何扰动强度、虹彩模式、输出透明度以及是否响应交互等参数来区分。

2.2 屏幕空间折射实现

2.2.1 折射方向

在 fragment shader 中，入射方向取视线方向 eye，法线取世界法线 normal。肥皂液近似水的折射率，程序使用

$$\eta = \frac{n_{\text{air}}}{n_{\text{film}}} = \frac{1.0}{1.33}.$$

GLSL 中调用 refract(eye, normal, eta) 得到折射方向，并转换到视图空间：

```
vec3 refractVec = mat3(vView) * refract(eye, normal, iorRatio)
;
```

随后取折射方向的 xy 分量作为屏幕空间偏移方向。因为这是屏幕空间近似，shader 并不计算真实光线与后表面的三维交点，而是把折射方向转化为背景纹理采样偏移。

2.2.2 边缘增强与薄膜厚度影响

肥皂泡的中心通常接近透明，轮廓处视觉效果更明显。程序用

$$e = 1 - |\mathbf{V} \cdot \mathbf{N}|$$

表示边缘程度，其中 $\mathbf{V}$ 是视线方向， $\mathbf{N}$ 是表面法线。随后通过 smoothstep 得到更平滑的边缘曲线：

$$e_{\text{profile}} = \text{smoothstep}(0.18, 0.92, e).$$

折射偏移的核心形式可写为：

$$\Delta \mathbf{p} = \mathbf{T}_{xy} s_{\text{ref}} s_{\text{face}} s_{\text{film}} r_{\text{px}} s_{\text{edge}} s_{\text{touch}},$$

其中 $\mathbf{T}_{xy}$ 是视图空间折射方向的屏幕分量， s_{ref} 是用户可调的 uRefractionStrength， s_{face} 区分正面和背面， s_{film} 由薄膜厚度和边缘程度共同决定， r_{px} 是泡泡屏幕半径， s_{edge} 是边缘增强。

背面 pass 使用更小的折射强度：

$$s_{\text{face}} = \begin{cases} 0.16, & \text{背面 pass,} \\ 1.0, & \text{正面 pass.} \end{cases}$$

这样做可以避免双界面叠加时背景被过度扭曲。最终偏移还会被 uMaxOffsetRatio 限制在泡泡屏幕半径的一定比例内，防止采样跳变。



图 2.2 背景小球经过主泡泡后的折射扭曲效果

2.3 薄膜虹彩模型

2.3.1 薄膜干涉基础

肥皂泡薄膜的厚度通常为数百纳米，和可见光波长处在同一数量级。光线在空气-薄膜界面与薄膜-空气界面发生反射，两个反射波之间产生光程差。对某一波长 λ ，相位差可写为：

$$\phi_\lambda = \frac{4\pi n d \cos \theta_t}{\lambda},$$

其中 n 是薄膜折射率， d 是薄膜厚度， θ_t 是薄膜内部折射角。不同波长的反射强度随 ϕ_λ 改变，因此 RGB 颜色会随厚度、视角和法线方向变化。

项目中所有虹彩模式都使用动态厚度 d 与视角项 $|\mathbf{V} \cdot \mathbf{N}|$ 作为输入，但三种模式计算反射率的方式不同。

2.3.2 Kim2012 模式：实时 RGB 波长采样

程序界面中的 **Kim2012** 模式对应 `uIridescenceMode = 0`。它参考 Kim 与 Park 关于实时肥皂泡虹彩渲染的做法^[2]，在 shader 中直接计算薄膜对 RGB 三个代表波长的反射率。实现文件为 `shader_refraction.h` 中的：

- `fresnelCoefficients`
- `interferencePhase`
- `thinFilmReflectance`
- `kim2012Iridescence`

对每个波长，程序先计算 s 偏振和 p 偏振的 Fresnel 振幅系数，再用相位差得到反射率近似：

$$R_\lambda = \frac{r_s^2(1 - \cos \phi)}{1 + r_s^4 - 2r_s^2 \cos \phi} + \frac{r_p^2(1 - \cos \phi)}{1 + r_p^4 - 2r_p^2 \cos \phi}.$$

当前版本采样的三个代表波长为：

$$\lambda_R = 615\text{nm}, \quad \lambda_G = 535\text{nm}, \quad \lambda_B = 465\text{nm}.$$

该模式的优点是公式直接、易解释，能清楚体现薄膜厚度和相位差对颜色的影响；不足是只用三个波长代表完整可见光谱，边缘颜色可能更鲜艳，也更容易出现强烈的 RGB 分离。

2.3.3 Belcour Airy 模式：多波长 Airy 反射近似

Belcour Airy 模式对应 `uIridescenceMode = 2`，也是当前默认模式。它参考 Belcour 与 Barla 对薄膜虹彩和 Airy reflectance 的建模思想^[3]，但本项目没有完整实现论文中的微表面解析预积分，而是在肥皂泡 `shader` 中实现了一个面向移动端实时展示的简化 Airy 反射模型。

对某一偏振方向，薄膜多次内反射的复振幅可写为：

$$A_\lambda = r_{12} + \frac{t_{12}r_{23}t_{21}e^{i\phi_\lambda}}{1 - r_{21}r_{23}e^{i\phi_\lambda}}, \quad R_\lambda = |A_\lambda|^2.$$

程序分别计算 s、p 两个偏振方向，再取平均。随后在 `belcourAiryWeightedIridescence` 中采样多个波长：

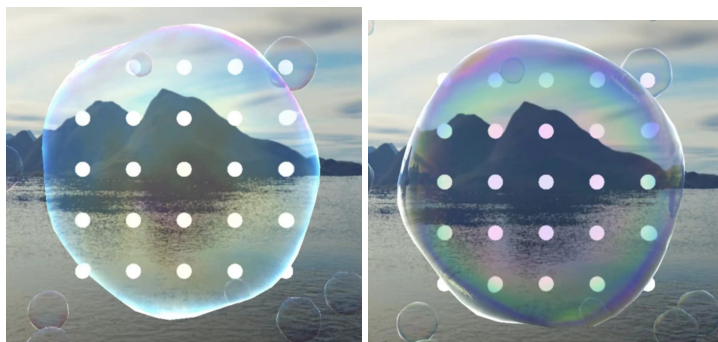
430, 470, 500, 535, 575, 615, 650nm.

这些波长再乘以经验 RGB 权重得到最终颜色。相比 Kim2012 模式，Belcour Airy 模式的颜色更平滑，环境反射也会被薄膜反射率染色，因此更适合作为最终鸿蒙版本的默认外观。

2.3.4 不同模式对比

表 2.1 不同虹彩模式的区别

模式	计算方式	优点	局限
Kim2012	<code>shader</code> 中实时计算 RGB 三个代表波长的薄膜干涉反射率。	公式直观，便于说明相位差、Fresnel 系数和厚度的关系。	只采样三个波长，颜色可能偏强，光谱连续性有限。
Belcour Airy	使用 Airy 风格的多次内反射公式，并采样多个波长加权合成。	颜色过渡更平滑，边缘和环境反射更自然，是当前默认模式。	是项目级简化实现，不是完整 Belcour 微表面预积分模型。



Kim2012

Belcour Airy

图 2.3 不同虹彩模式对比

2.4 动态膜厚与颜色合成

2.4.1 动态膜厚

真实肥皂泡的膜厚会因为重力排液、表面流动和扰动而变化。完整的膜厚动力学可以基于表面上的流体方程建模^[4]。本项目为了满足移动端实时演示要求，采用 shader 中的程序化近似：使用世界法线方向作为膜面参数，叠加多层 Simplex noise、缓慢流动项、细节噪声和上下方向的排液趋势。

动态厚度可概括为：

$$d = d_0 + A_{\text{var}}P(\mathbf{n}, t) + 190M_{\text{touch}} + 95Q_{\text{ripple}},$$

其中 d_0 是基础膜厚， A_{var} 是膜厚变化幅度， $P(\mathbf{n}, t)$ 是由多层噪声和排液项组合的厚度图案，其中 d_0 是基础膜厚， A_{var} 是膜厚变化幅度， $P(\mathbf{n}, t)$ 是由多层噪声和排液项组合的厚度图案。

一个重要实现细节是：项目使用 `normalize(vWorldNormal)` 作为噪声输入方向，而不是使用 UV 坐标。这样可以避免球体 UV 接缝处出现明显断裂，让虹彩流动更连续。

2.4.2 颜色合成

泡泡最终颜色由三部分组成：

1. **透射背景**：从背景 FBO 或背面 FBO 中折射采样得到 `bgColor`。
2. **薄膜虹彩**：由当前模式计算得到 `filmReflectance`，再根据 Fresnel 边缘项增强。
3. **环境反射和高光**：通过 `cubemap` 采样环境反射，并用薄膜反射率染色；另外加入少量 Blinn-Phong 高光。

中心区域透明度更高，轮廓处透明度下降、反射和虹彩增强。shader 中用

$$F = (1 - |\mathbf{V} \cdot \mathbf{N}|)^p$$

作为 Fresnel 边缘项，其中 p 对应可调参数 `uFresnelPower`。这使泡泡中心能保持清透，边缘能呈现更明显的彩色轮廓。

第3章 物理模拟与动力学

肥皂泡的本质是浸没在环境空气中的一个极薄液膜，其动力学行为主要由液膜的表面张力、周围空气的不可压缩性，以及空气的惯性主导。传统的基于体积的多相纳维-斯托克斯（Navier-Stokes）求解器虽然能够精确模拟这些现象，但需要大量的计算资源来解析泡泡内部的空气动力学以及薄至微米级的薄膜几何形状。

本项目的方法参考 Da 等人提出的 *Double Bubbles Sans Toil and Trouble: Discrete Circulation-Preserving Vortex Sheets for Soap Films and Foams*^[5] 的思路，仅将建模工作集中在泡泡表面。该论文首次提出了一种基于非流形涡流片（vortex sheet）方程来模拟肥皂膜、泡泡和泡沫动力学的数值方法。本项目在其理论上实现了一个简化版本，专门用于单个闭合泡泡的动态表面演化。

3.1 涡流片模型的理论基础

涡流片是一个浸没在三维空间中的表面，它界定了速度场切向分量的不连续性——穿过薄片时，速度的切向分量发生跳跃，而法向分量保持连续。这一数学结构恰好契合肥皂膜的物理本质：薄膜两侧分别是内部空气和外部空气，两侧的速度场可以不同，但空气不能穿透薄膜。

DBSTT 方法使用一个标量场——**环量（circulation）** Γ ——作为描述涡流片状态的主要变量。对于表面上的每一个点 $f(X)$ ，其环量值定义为沿着一个从固定参考点 x_0 到该点、且穿透薄片的任意闭合环路的环路积分：

$$\Gamma(X) = \oint_L \mathbf{u} \cdot d\mathbf{x} \quad (3.1)$$

其中 $\mathbf{u}$ 为环境空气的速度场。这一积分的值与环路 L 的具体形状无关（只要它仅在 x_0 和 $f(X)$ 两处穿透薄片），这保证了 Γ 是表面上的良定义的标量场。

给定环量场之后，可以分两步恢复速度场。首先计算涡流片强度 γ ：

$$\gamma = \mathbf{n} \times \nabla_f \Gamma \quad (3.2)$$

其中 $\mathbf{n}$ 为表面单位法线， ∇_f 为表面梯度算子。

接着，通过 Biot-Savart 积分恢复环境速度场：

$$\mathbf{u}(\mathbf{x}) = \frac{1}{4\pi} \int_S \gamma(\mathbf{x}') \times \frac{\mathbf{x} - \mathbf{x}'}{|\mathbf{x} - \mathbf{x}'|^3} df' \quad (3.3)$$

$\mathbf{u}$ 自动满足 $\nabla \cdot \mathbf{u} = 0$ 。这意味着基于涡流片运动学导出的速度场能够自然地捕捉不可压缩流体的动力学特征，而无需像传统网格方法那样在每步显式求解压力-泊松方程来强制不可压缩性。

下面考虑表面张力驱动的环境演化。在不受外力作用时，根据开尔文环量定理，环量随物质平流守恒： $D\Gamma/Dt = 0$ 。而由于表面张力的存在，环量场会按照一个简洁的局部定律演化。

考虑不可压缩欧拉方程在涡流片两侧的极限形式，结合表面张力产生的压力跳跃 $p_1 - p_2 = \sigma(H_1 - H_2)$ （其中 σ 为表面张力系数， H_1, H_2 分别为薄膜两侧的有符号标量平均曲率），可以得到环量演化的核心方程：

$$\frac{D\Gamma}{Dt} = -\frac{\sigma}{\rho}(H_1 - H_2) \quad (3.4)$$

其中 ρ 为空气密度。对于闭合的单个泡泡，两侧的几何重合但朝向相反，因此 $H_1 = -H_2$ ，从而 $H_1 - H_2 = 2H$ 。这意味着**环量的时间变化率与表面的有符号标量平均曲率成正比**——表面曲率大的区域环量变化快，曲率小的区域变化慢。表面张力正是通过这一机制驱动泡泡表面趋向平衡态（对闭合泡泡而言是球面，其上各点曲率相等）。

3.2 离散化与数值实现

将上述连续理论转化为数值算法，需要在空间和时间维度上进行离散化。以下详细介绍本项目在 `VortexSheetSimulation` 类中的实现。

使用三角网格来表示泡泡表面，在顶点上采样环量场。项目定义了 `SimMesh` 结构来承载模拟所需的所有数据：

- `positions`: 顶点三维位置 $\mathbf{p}_i \in \mathbb{R}^3$
- `triangles`: 三角形索引三元组 (i_0, i_1, i_2)
- `circulation`: 逐顶点存储的标量环量 $\Gamma_i \in \mathbb{R}$
- `vertexNormals`: 顶点法线 $\mathbf{n}_i$ （通过角度加权平均求出）
- `voronoiAreas`: 顶点 Voronoi 对偶面积 A_i
- `meanCurvature`: 逐顶点有符号标量平均曲率 H_i
- `velocity`: 逐顶点的诱导速度 $\mathbf{u}_i$

初始几何由 `initIcosphere()` 构建：程序首先生成一个单位二十面体，再通过 `midpoint` 方式对 `icosphere` 做指定次数的细分，最后归一化到指定半径 $R = 1.5$ 。为了触发动力学演化，程序沿一个固定拉伸轴施加初始顶点扰动；在当前鸿蒙提交版本中，应用层设置的扰动幅度为 16%。环量场初始化为全零，法线和面积在初始化完成后立即计算。

算法总览

每次仿真更新遵循 DBSTT 论文^[5] Algorithm 1 的时间积分循环，经过本项目简化适配后的子步更新流程为：

1. 几何量计算: `computeVertexNormalsAndAreas()` + `computeMeanCurvature()`
2. 表面张力积分: `integrateSurfaceTension(dt)`
3. 环量扩散: `diffuseCirculation()`
4. 涡流片强度计算: `computeVortexSheetStrength(gamma)`
5. 毕奥-萨伐尔速度求解: `computeBiotSavartVelocities(gamma)`
6. 位置推进: `advectPositions(dt)`

`VortexSheetSimulation` 类内部保留了 `substepsPerFrame = 5`、`timeStep = 0.0005f` 的默认配置，但当前鸿蒙提交版本在初始化与重置时都会显式改写为 `substepsPerFrame = 1`、`timeStep = 0.001f`。同时，Native 渲染层并不是每一帧都推进仿真，而是每隔 `kSimFrameInterval = 6` 帧调用一次 `g_Sim.update(1.0f / 60.0f)`。因此，当前系统中一次实际仿真更新只执行 1 个子步，子步时间步长为 $\Delta t = 0.001$ 秒，子步结束后重新计算法线以供渲染使用。

顶点法线与 Voronoi 面积计算

顶点法线采用角度加权平均法计算。对每个入射到顶点 v 的三角形面，计算该面的单位法线 $\mathbf{n}_f$ ，并将其按该三角形在顶点处的内角加权累加：

$$\mathbf{n}_v = \frac{\sum_{f \in \text{faces}(v)} \alpha_f \cdot \mathbf{n}_f}{\|\sum_{f \in \text{faces}(v)} \alpha_f \cdot \mathbf{n}_f\|} \quad (3.5)$$

其中 α_f 为面 f 中位于顶点 v 处的内角，通过相邻边向量的点积与反余弦求得。对于退化情况（法线长度接近于零），则退化为从原点指向顶点的径向方向。

Voronoi 面积通过将每个入射三角形的面积三等分并累加到其三个顶点上来近似计算。

离散平均曲率

对于标量平均曲率的离散化，本项目参考论文的建议，采用基于边的离散曲率公式：对每对相邻顶点 (v_i, v_j) （即每条共享两个三角形的边），计算该边上的积分平均曲率，再均匀分配到你两个端点。具体来说，对于连接 v_i, v_j 的边，假设它由两个三角形 t_0, t_1 共享：

$$\mathbf{K}_i = -\frac{1}{4A_i} \sum_{j \in \mathcal{N}(i)} (\cot \alpha_{ij} + \cot \beta_{ij}) (\mathbf{p}_j - \mathbf{p}_i) \quad (3.6)$$

其中 A_i 为顶点 v_i 的 Voronoi 面积， $\mathcal{N}(i)$ 为 v_i 的一环邻域， α_{ij}, β_{ij} 分别为边

e_{ij} 在两个入射三角形中对面的两个角。 $\cot \alpha$ 通过边向量的点积除以叉积模长求得：

$$\cot \alpha = \frac{\mathbf{e}_{ik} \cdot \mathbf{e}_{jk}}{\|\mathbf{e}_{ik} \times \mathbf{e}_{jk}\|} \quad (3.7)$$

在代码实现中，程序首先构建从边到其入射面列表的映射，然后仅处理恰好由两个三角形共享的边，计算两侧的余切权重并累加到曲率法线 $\mathbf{K}_i$ 上。最终的标量平均曲率通过对曲率法线取负点积获得：

$$H_i = -\mathbf{K}_i \cdot \mathbf{n}_i \quad (3.8)$$

负号确保了向外法线约定下的符号正确性：在凸区域（如球面）， $\mathbf{K}$ 指向内侧，与向外法线 $\mathbf{n}$ 的点积为负，取负后 $H > 0$ 。

表面张力积分

有了每个顶点的离散标量平均曲率 H_i ，就可以根据方程 (3.4) 对环量进行前向欧拉更新。对于单个闭合泡泡，两侧界面几何重合但朝向相反： $H_1 = H, H_2 = -H$ ，故 $H_1 - H_2 = 2H$ 。离散更新公式为：

$$\Delta \Gamma_i = \text{sign} \cdot \frac{\sigma}{\rho} \cdot \Delta t \cdot 2 \cdot H_i \quad (3.9)$$

其中 σ/ρ 对应代码中的 `surfaceTensionStrength`（默认值 15.0），`sign` 通过 `flipSurfaceTensionSign`（默认 `true`，即取正号）控制。

值得注意的是，论文中原本的公式包含顶点面积 A_v 在分母中，但由于我们使用逐点的 H_i 而非积分的曲率 H_i^v ， A_v 在分子分母中恰好抵消简化。

Vortex Sheet Strength 计算

环量更新完成后，需要在每个三角形上计算涡流片强度 γ 。根据方程 (3.2)， $\gamma = \mathbf{n} \times \nabla_f \Gamma$ 。在三角形 $t = (v_0, v_1, v_2)$ 上，环量场是分段线性的，其表面梯度为常数。代码实现使用三角形三个顶点的环量值来计算梯度：

$$\gamma_t = \mathbf{n}_t \times \frac{\Gamma_0(\mathbf{p}_2 - \mathbf{p}_1) + \Gamma_1(\mathbf{p}_0 - \mathbf{p}_2) + \Gamma_2(\mathbf{p}_1 - \mathbf{p}_0)}{2 \cdot \text{area}(t)} \quad (3.10)$$

其中 $\mathbf{n}_t$ 为三角形面的单位法线，通过边向量的叉积归一化得到，且保证与从原点指向三角形质心的方向一致（向外朝向）。

正则化毕奥-萨伐尔速度求解

对于每个顶点 v_i ，需要计算所有三角形 t_j 对其诱导速度的贡献之和。为避免距离趋近于零时的奇异性，DBSTT 采用正则化形式：

$$\mathbf{u}(\mathbf{x}) = \frac{1}{4\pi} \sum_t \gamma_t \times \frac{\mathbf{x} - \mathbf{c}_t}{(|\mathbf{x} - \mathbf{c}_t|^2 + \alpha^2)^{3/2}} \cdot A_t \quad (3.11)$$

其中 $\mathbf{c}_t$ 为三角形 t 的质心， A_t 为其面积， α 为正则化参数。代码中 α 自动设为网格平均边长（在初始化时计算）的 1.0 倍，即 `regularizationAlpha = avgEdgeLength * 1.0f`。

实现细节上，程序预计算每个三角形的质心、面积和 γ_t 值，然后对每个顶点遍历所有三角形，进行双重循环求和。每个贡献项为 $\gamma_t \times \mathbf{r}$ 乘以面积和分母幂次的倒数，其中 $\mathbf{r} = \mathbf{x} - \mathbf{c}_t$ 为从三角形质心到当前顶点的向量。最终速度除以 4π 以匹配连续积分中的系数。

位置推进与安全钳制

顶点位置使用前向欧拉法更新： $\mathbf{p}_i \leftarrow \mathbf{p}_i + \mathbf{u}_i \cdot \Delta t$ 。为确保稳定性，代码施加了一个位移上限，限制每个子步的顶点位移不超过平均边长的 10%：

$$\|\Delta \mathbf{p}_i\|_{\max} = 0.1 \cdot \bar{e} \quad (3.12)$$

其中 $\bar{e}$ 为平均边长。这是为了解决调试代码过程中出现的因数值不稳定导致的网格穿刺或顶点飞散问题。

位置更新后，所有顶点统一减去网格质心，以防止因数值漂移导致的整体平移，保持毕奥-萨伐尔积分的对称性。

环量扩散

为了进一步抑制数值不稳定性，代码在每个子步中施加一个轻微的拉普拉斯扩散：

$$\Gamma_i \leftarrow \Gamma_i + \nu \cdot \frac{1}{|\mathcal{N}(i)|} \sum_{j \in \mathcal{N}(i)} (\Gamma_j - \Gamma_i) \quad (3.13)$$

其中 ν 为扩散系数，默认值 `circulationDiffusion = 0.0005`，非常小，仅用于消除高频数值振荡而不影响宏观动力学行为。DBSTT 论文也指出，这种扩散可以解释为对空气粘度的合理视觉近似。

3.3 与本项目渲染管线的集成

物理模拟与渲染管线的衔接位于 `RenderFrame()` 函数中：

1. 渲染循环开始后，若模拟未被暂停（`g_SimPaused == false`），则每隔 `kSimFrameInterval = 6` 帧调用一次 `g_Sim.update(1.0f/60.0f)`，按较低频率推进一次仿真更新。

2. 模拟完成后, 调用 `buildVerticesFromSim()` 将 `SimMesh` 中的顶点位置和法线转换为 GPU 可用的 `Vertex` 数组。
3. 通过 `Mesh::updateVertices()` 将更新后的顶点数据上传至 `OpenGL VBO`, 后续的折射/虹彩 `shader` 即可使用变形后的几何体进行渲染。

主泡泡的模型矩阵保持为单位矩阵, 所有变形信息完全包含在经模拟更新的顶点位置中, 这使得 `shader` 中的法线计算能够正确地反映泡泡表面的动态形变。

当前实现没有加入原论文中的完整非流形拓扑变化、泡沫多区域标记和线框约束, 只是一个 `DBSTT` 风格的单泡泡表面动态近似。它的作用是让主泡泡轮廓和表面法线随时间发生微小变化, 从而让折射和虹彩不显得完全静止。

具体动态效果详见项目演示视频。可以明显观察到主泡泡在当前系统中保持持续的小幅表面形变, 并在参数调整后呈现不同强度的恢复趋势。

第4章 多泡泡交互系统

4.1 系统概述

决赛阶段，系统从初赛的”单主泡泡 + 16 个装饰展示泡泡”架构演进为完整的动态多泡泡交互系统。新系统支持运行时动态创建与删除泡泡、泡泡间接触检测与响应、共享膜的形成与演化、以及三种接触结果（分离、稳定 8 字双泡、融合吸收）。本章按照数据结构、空间加速、接触动力学、表面变形、共享膜几何、结果选择与辅助系统的顺序，介绍多泡泡交互的核心设计。

与初赛版本的关键区别在于：

1. 每个泡泡不再是静态的展示实例，而是一个拥有独立身份（BubbleId）、物理状态和持久几何表示的仿真对象。
2. 泡泡之间的接触关系由统一的接触图（contact graph）管理，每个泡泡可以同时参与多条接触关系。
3. 接触过程不是单帧状态切换，而是通过连续激活函数驱动从接近、预接触、成膜到结果锁定的平滑过渡。
4. 泡泡表面不再是纯球面，而是通过 26 个持久控制点进行局部变形，使接触区域的压扁、鼓包和回弹具有记忆性和惯性。

本章的实现参考了多篇多泡泡/泡沫仿真的研究工作。接触图的构建借鉴了^[6]基于加权 Voronoi 图的邻接思想；表面控制点的弹簧-阻尼动力学借鉴了 MELP^[7]将局部粒子与全局几何结合的思路；弯曲共享膜的 Young-Laplace 曲率参考了多尺度泡沫模型^[8]中的压力差关系；体积守恒策略参考了^[9]的体积控制方法。需要强调的是，本系统的实现是面向实时可视化的工程近似，并非对上述论文算法的完整复现。

4.2 核心数据结构

4.2.1 泡泡对象：DisplayBubble

每个泡泡由 DisplayBubble 结构体表示，其核心字段包括：

- id: 稳定的 64 位泡泡标识（BubbleId），不随数组重排而改变。
- position、velocity: 泡泡中心的运动状态。
- radius、initialRadius: 当前半径与初始半径，半径可随时间向目标体积恢复。
- targetVolume: 目标体积，由初始半径计算得出，用于体积守恒校正。
- basePosition: 场景中的松弛位置，自由状态下泡泡围绕此位置做低频

摆动。

- phase、windAmplitude、floatAmplitude、speed: 自由漂移参数。
- filmThickness: 泡泡膜厚状态。
- surfaceControls: 26 个持久表面控制点的数组。
- state: 泡泡当前状态 (Free、Touching、SharedFilm、Merged 等)。

4.2.2 接触关系: BubbleContactPair

每一对候选或已接触的泡泡由 BubbleContactPair 管理, 核心字段包括:

- a、b: 两个泡泡的稳定 BubbleId。
- candidate: 是否已进入近场候选范围。
- bonded: 是否已建立持久的共享膜关系。
- contactActivation: 连续接触激活强度, 从 0 到 1 平滑增长。
- contactTime: 接触历史累计时间。
- contactRadius、contactRadiusVelocity: 接触圆半径及其变化速率。
- interactionCompression、interactionVelocity: 交互压缩程度及其速率。
- filteredNormal: 滤波后的接触轴方向 (从 A 指向 B)。
- filteredPlaneCenter: 滤波后的接触平面中心。
- filmThickness: 该接触对的共享膜简化厚度状态。
- bridgeStrength、geometryBlend: 接触几何的成熟程度。
- state: 该接触对的阶段 (Touch、SharedFilm、Merged 等)。

接触图以泡泡为节点、BubbleContactPair 为边, 支持一个泡泡同时参与多条接触关系。例如三个泡泡可同时存在 AB、AC、BC 三条接触边, 每条边独立维护自己的接触圆和共享膜参数。

4.2.3 表面控制点: SurfaceControl

每个泡泡维护 26 个低分辨率表面控制点。控制方向来自 $\{-1, 0, 1\}^3$ 中除原点外的 26 个方向, 均匀覆盖球面。每个控制点存储:

- localDir: 控制点在单位球面上的参考方向。
- displacement: 相对于自由球面的持久三维位移。
- velocity: 该局部表面的变形速率。

控制点使表面变形具有记忆、惯性和恢复过程。控制点对渲染顶点的插值权

重近似为：

$$w_{ik} = \max(\mathbf{d}_i \cdot \mathbf{c}_k, 0)^8,$$

其中 $\mathbf{d}_i$ 为渲染顶点在单位球面上的方向， $\mathbf{c}_k$ 为第 k 个控制点的方向。所有控制点按权重归一化插值得到最终的持久位移。

4.3 空间哈希与 Broad Phase

为避免泡泡数量增加后对所有泡泡对执行 $O(N^2)$ 距离检查，系统采用三维均匀空间哈希网格进行 **broad phase** 加速。网格尺寸约为最大泡泡直径的 1.2 倍。每帧仅检查当前泡泡所在网格及其 26 个相邻网格中的其他泡泡，从而快速生成候选接触对。

该步骤称为 **broad phase**，只负责生成候选 **pair**，不涉及具体的接触几何计算。

4.4 接触激活与连续过渡

两泡泡表面间距定义为：

$$g = \|\mathbf{p}_A - \mathbf{p}_B\| - (r_A + r_B).$$

当间距 g 进入近场范围 **nearRange**（约 $0.08 \cdot \min(r_A, r_B)$ ）时，接触激活强度开始连续增长：

$$a = \text{Smooth01}\left(\frac{\text{nearRange} - g}{\text{nearRange}}\right),$$

其中 $\text{Smooth01}(x) = x^2(3 - 2x)$ ，输入被限制在 $[0, 1]$ 内。该激活强度从接近阶段连续增长，并统一驱动以下所有效果：

- 精细吸引力（毛细节 **attraction**）；
- 法向速度阻尼；
- 预接触局部压扁；
- 接触圆初始半径；
- 接触平面和法线滤波；
- 球壳 **clipping** 裁剪；
- 共享膜可见度；
- 表面控制点的局部压缩。

这种设计确保从 **candidate** 到 **bonded** 的过渡不会在单帧内突然切换一整套视觉逻辑。当 a 超过阈值且间距足够小时，系统建立 **bonded contact**，这时候选帧

的进度和入射速度被继承。

4.5 接触圆动力学与位置约束

4.5.1 接触圆半径

`contactRadius` 表示球壳与共享膜相交的圆形边界半径。它通过弹簧-阻尼系统追踪几何目标值：

$$\ddot{r}_c = k_r(r_c^{\text{target}} - r_c) - d_r\dot{r}_c,$$

其中目标值 r_c^{target} 由当前泡泡中心距离和两泡泡半径的几何关系反推。参数选取接近临界阻尼，使接触圆快速增大但避免过冲回弹。

接触圆增大后，两泡泡的目标中心距离由几何关系反推：

$$d_{\text{target}} = \sqrt{r_A^2 - r_c^2} + \sqrt{r_B^2 - r_c^2}.$$

当 $r_c = 0$ 时目标距离即为 $r_A + r_B$ ；接触圆越大，两中心允许靠得越近，但相向球冠会被裁剪掉。

4.5.2 多接触位置约束迭代

系统每帧执行 6 次接触图位置约束迭代。每条接触边根据当前距离与目标距离的偏差修正两个泡泡的中心位置。多次迭代的作用是让多条约束共同收敛。多接触变形还按活跃接触数量进行分摊。

4.6 持久表面控制点系统

4.6.1 弹簧-阻尼更新

每帧首先为每个 `SurfaceControl` 生成自由振荡目标，然后聚合该泡泡参与的所有接触的影响。控制点采用弹簧-阻尼更新：

$$\ddot{\mathbf{d}}_k = k_s(\mathbf{d}_k^{\text{target}} - \mathbf{d}_k) - d_s\dot{\mathbf{d}}_k.$$

4.6.2 接触区域的变形目标

接触方向内部使用 `capMask` (帽区遮罩)，接触圆边缘使用 `rimMask` (环区遮罩)：

$$\mathbf{d}_k^{\text{contact}} = -\mathbf{n}_{\text{contact}} \cdot c_{\text{comp}} \cdot M_{\text{cap}} + \mathbf{r}_{\text{radial}} \cdot c_{\text{bulge}} \cdot M_{\text{rim}},$$

其中 $\mathbf{n}_{\text{contact}}$ 为接触轴方向， $\mathbf{r}_{\text{radial}}$ 为远离接触轴的径向方向。含义为：

- 接触帽内部沿接触轴向内压缩；
- 接触圆边缘向侧面鼓起；
- 多条 **contact** 的目标按权重聚合。

接触区域使用更高刚度和适中的欠阻尼，使表面快速响应但仍保留软膜惯性。

4.7 接触区域速度共享

借鉴 MELP^[7] 的 matching-velocity 思想，系统对两侧接触帽计算表面材料速度：

$$\mathbf{v}_{\text{surface}} = \mathbf{v}_{\text{center}} + \boldsymbol{\omega}_{\text{control}} \cdot \mathbf{r},$$

然后求质量加权的共享速度，让两侧接触帽局部趋向该速度。这并非将两个泡泡整体锁定，而是让共享膜附近的局部表面一起运动。每条 **pair** 独立计算自己的帽区权重。

4.8 共享膜渲染

每条 **candidate** 或 **bonded contact** 绘制一张共享膜圆盘。共享膜与球壳使用相同的 `contactRadius`、`filteredPlaneCenter` 和 `filteredNormal`，因此球壳裁剪边界和共享膜边界由同一套接触状态驱动。

共享膜采用较低的 **alpha**、折射强度和環境反射强度，以减少多个透明 **mesh** 叠加产生的黑带效应。



图 4.1 接触泡泡之间的共享膜渲染

4.9 弯曲共享膜与 Young-Laplace 压力

等尺寸泡泡之间的共享膜近似为平面，但不同尺寸泡泡之间存在内部压力差。根据 Young-Laplace 关系，膜曲率与两侧压力差成正比：

$$\kappa = \frac{1}{r_A} - \frac{1}{r_B},$$

其中法线方向约定为从 A 指向 B。于是：

- 两泡半径相同时 $\kappa = 0$ ，共享膜退化为平面；
- A 更大时膜向 A 一侧弯曲（A 内部压力更低）；
- B 更大时膜向 B 一侧弯曲。

共享膜使用解析球冠表示：

$$z(\rho) = \text{sign}(\kappa) \cdot \left(\sqrt{R^2 - \rho^2} - \sqrt{R^2 - r_c^2} \right),$$

其中 $R = 1/|\kappa|$ ， ρ 为到接触圆中心的径向距离。当 $\rho = r_c$ 时 $z = 0$ ，弯曲膜边界始终位于原接触平面，不因增加曲率产生位置缝隙。

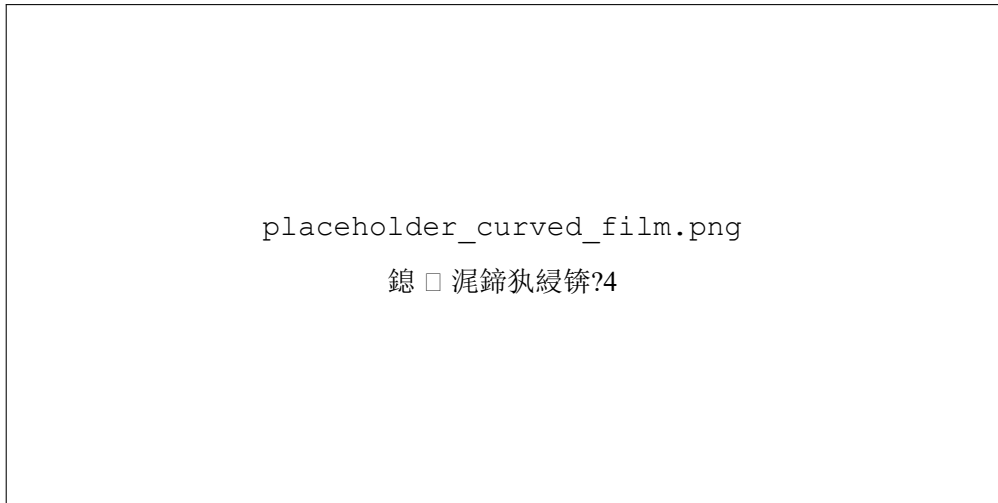


图 4.2 不同尺寸泡泡间的弯曲共享膜：膜向大泡一侧凸出

4.10 接触结果选择

接触成熟后，系统根据以下因素计算概率并锁定一次结果：

- $\min(r_A, r_B) / \max(r_A, r_B)$ ：尺寸比；
- 接触持续时间；
- 共享膜厚度状态；
- 接触圆相对于小泡泡的尺寸；
- 法向碰撞速度；
- 当前风场扰动。

泡泡 ID 和当前场景代号共同生成稳定随机值，因此结果不随帧率抖动。三种结果如下。

4.10.1 分离

分离路径不会在首次碰撞时立即弹开，而是经历以下过程：

1. 两泡泡短暂压缩并形成共享膜；
2. 共享膜和接触圆逐渐收缩；
3. 粘结合约被释放；
4. 沿接触法线施加柔性分离速度；
5. 两泡泡恢复独立运动。

接触面积较小或冲击较强时，分离概率会提高。普通交互的分离概率限制在 2% 到 8% 之间。

4.10.2 稳定8字双泡

尺寸接近的泡泡更倾向形成稳定8字双泡：

- 两泡泡仍是独立腔体；
- 外轮廓形成8字形；
- 中间保留共享膜，而非成为单腔花生形泡泡；
- 等尺寸时共享膜接近平面，不等尺寸时共享膜按 Young-Laplace 关系弯曲；
- 接触圆有轻微周期波动，使双泡不显完全静止。

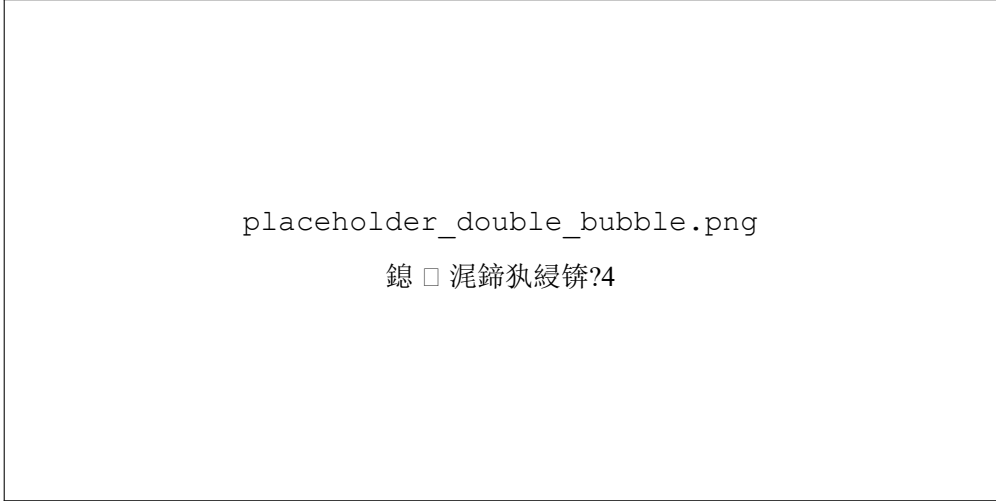


图 4.3 稳定8字双泡：等尺寸泡泡间的平面共享膜

4.10.3 融合

尺寸差越大，越倾向小泡泡并入大泡泡。融合过程为：

1. 共享膜逐渐变薄；
2. 接触膜局部破裂并扩大开口；
3. 外表面颈部扩张；
4. 两个外壳过渡为一张连续融合网格；
5. 融合网格被提升为幸存泡泡的持久表面。

融合中心采用体积加权：

$$\mathbf{p}_{\text{merged}} = \frac{\mathbf{p}_A V_A + \mathbf{p}_B V_B}{V_A + V_B},$$

最终半径保持体积守恒：

$$r_{\text{merged}}^3 = r_A^3 + r_B^3.$$

因此尺寸差较大时，最终中心更靠近大泡泡，小泡泡视觉上向大泡泡一侧被吸收。融合完成后移除被吸收泡泡的接触关系，幸存泡泡继承体积加权后的合速度。

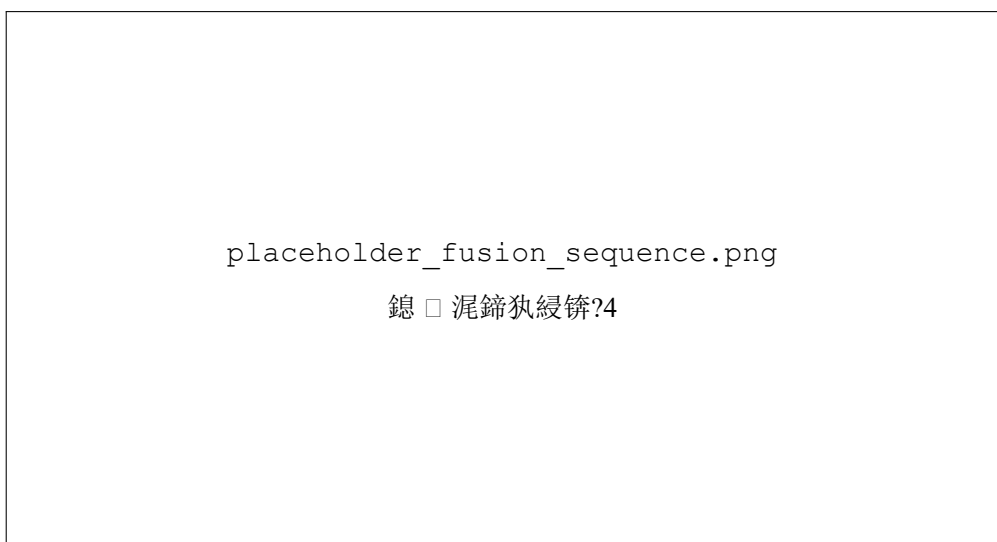


图 4.4 融合过程：小泡泡逐渐被大泡泡吸收

4.11 风场系统

全局风场默认关闭（界面 Wind 按钮切换）。核心特征：

- 风力强度范围 0.0 到 0.45，默认 0.16，单次调整步长 0.025；
- 风向通过界面滑块或按钮上下左右调整；
- 小泡泡具有更高的风力响应系数，大泡泡运动相对稳定；
- 关闭风后，已有速度通过阻尼平滑衰减，泡泡逐渐回到有界摆动范围；
- 融合后的泡泡也可整体随风移动。

无风状态下，泡泡并非完全静止，而是围绕各自的 `basePosition` 做有界低频摆动，使用不同相位和速度以避免同步运动。

4.12 每泡 DBSTT 涡流片仿真

每个展示泡泡在创建时即分配一个独立的 `VortexSheetSimulation` 实例，存储于全局映射 `g_BubbleSims` 中。仿真以单位半径、细分等级 2 的正二十面体网格创建（模型矩阵负责视觉缩放），初始表面张力系数为 15.0，子步数为 3，时间步长 0.001 秒。

在正常运行（非破裂）状态下，该仿真实例处于休眠——`UpdateBubbleSims()` 仅对 Burst 状态的泡泡调用 `sim->update()`。泡泡的常规表面形变仍由 26 个持久控制点的弹簧-阻尼系统（`BuildPersistentBubbleVertices`）驱动，接触、融合和演示场景不受影响。这一设计避免了为每个泡泡每帧都执行 $O(N^2)$ 的 Biot-Savart 积分，只在破裂这一短暂事件中激活物理求解器。

当泡泡进入破裂状态时，仿真实例被重新初始化为泡泡的实际半径，并执行开洞操作（见下文），随后提升表面张力参数驱动膜的物理回缩。这使得破裂时膜的变形和回缩由 DBSTT 涡流片模型^[5]实时计算，而非预设的关键帧动画。

4.13 泡泡破裂机制

系统支持运行时通过触控点击触发泡泡破裂，模拟真实肥皂泡被戳破后的膜回缩行为。破裂机制由 `TriggerBubbleBurst()` 函数实现，核心流程如下：

1. **触发方式：**在 HarmonyOS 端通过界面 Pop 模式触发：点击目标泡泡即启动破裂动画。该模式保持激活直至用户切换到其他模式。
2. **状态切换：**目标泡泡状态变为 `Burst`，破裂持续时间设为 0.35 秒，破裂缩放系数初始化为 1.0。
3. **DBSTT 物理回缩：**以泡泡实际半径重新初始化涡流片仿真网格，然后沿相机方向执行开洞操作（`punchHole`），在膜上制造一个约 26° 张角的自由边缘供表面张力回缩。随后将表面张力系数从默认的 15.0 提升至 120.0，子步数从 3 增至 12，时间步长调整为 0.002 秒，环量扩散系数降至 0.0001 以减少能量耗散。开洞后的仿真三角网格通过 `Mesh::updateGeometry()` 直接替换渲染网格（顶点数从 2057 降至约 150），此后每帧由 `UpdateMeshFromSim()` 将仿真顶点位置和法线上传至 GPU，使回缩过程完全由 DBSTT 物理模型驱动。
4. **开洞算法：**`punchHole(direction, threshold)` 遍历所有三角形，计算每个三角形质心方向与开洞方向的点积，删除点积超过 $\cos(\text{threshold})$ 的三角形。随后对保留的三角形进行顶点重映射，裁掉不再被任何三角形引用的孤立顶点，重建法线、Voronoi 面积、平均边长和 Biot-Savart 正则化参数。孔洞边界边在后续的平均曲率计算中因入射面数不等于 2 而被自然跳过，使孔洞边缘顶点不受表面张力直接驱动，回缩从孔洞附近的内部顶点开始向外传播。
5. **接触解耦：**立即停用该泡泡参与的所有接触对，将 `bonded`、`candidate`、`active`、`fusionActive`、`fusionComplete`、`persistentRenderPair` 全部置为 `false`，并将 `contactRadius`、`contactRadiusVelocity`、`geometryBlend`、`interactionCompression`、`contactActivation`、`bridgeStrength` 归零。这确保共享膜、融合表面和接触几何立即停止渲染，破裂泡泡不再受任何接触约束。

6. **冲击波传播**: 对破裂泡泡周围 4 倍半径范围内的其他泡泡施加向外的冲击波脉冲（衰减系数随距离线性下降），模拟真实泡泡破裂时释放的压力波。若邻近泡泡曾与破裂泡泡处于接触状态，还会获得额外的膨胀补偿（半径增加约 $0.04 \times 1.15 \times \text{falloff}$ ），因为原本约束它们的共享膜已消失。
7. **回缩动画与回退**: 在 0.35 秒内，涡流片仿真驱动膜从孔洞边缘向内回缩，透明度通过 smoothstep 曲线 $1.0 - \text{Smooth01}(t)$ 淡出（非线性，前半段保持较高可见度）。若泡泡没有可用的 DBSTT 仿真实例（例如某些代码路径未调用 `InitBubbleSim`），则回退为手动缩放：`burstScale` 按 smoothstep 从 1.0 降至 0，模型矩阵应用该缩放系数，透明度同样以 smoothstep 淡出。

破裂过程的视觉效果由 DBSTT 涡流片求解器^[5]实时计算，膜的变形和回缩速度由表面张力、涡流片强度（Biot-Savart 速度场）和扩散系数共同决定，而非预设的关键帧动画。这一设计使得每次破裂的表现受泡泡当前几何状态（半径、法线、接触历史）影响而略有差异，增强了视觉真实感。

4.13.1 渲染管线的多重防护

当两个贴在一起的泡泡（共享膜或 8 字双泡状态）中一个被点击破裂时，如果接触关系和共享膜没有被彻底清除，幸存泡泡的 shell 仍会被接触平面裁切，导致视觉上出现“两个泡泡隔着空气墙”的错误。为避免此类问题，系统在渲染管线的多个环节添加了针对 Burst 和 Dead 状态的防护：

- **接触平面裁切**: `SetBubbleContactUniforms()` 对 Burst 状态的泡泡直接返回 `uVisualContactCount = 0`，不向 shader 传递任何接触平面，避免破裂的 sim 网格被旧接触平面错误裁切。同时对幸存泡泡，跳过对端为 Burst/Dead 的接触对。
- **共享膜绘制**: `DrawContactBridges` 和 `DrawFusionSurfaces` 在遍历接触对时，若任一端处于 Burst 或 Dead 状态则跳过，确保共享膜和融合表面不会在破裂后继续渲染。
- **Shell 透明度**: 渲染循环对 Burst 状态的泡泡跳过 `alpha *= 1.0 - fusionSurfaceVisibility` 这一步，保证破裂的 shell 始终可见，不被残留的融合表面可见度吃掉。
- **Broad phase 隔离**: `UpdateDisplayBubbleInteractions` 的空间哈希 broad phase 跳过 Burst/Dead 状态的泡泡，不会在破裂泡泡被移除前重新为其创建候选接触对。
- **模型矩阵**: `PersistentBubbleModelMatrix` 对有 sim 实例的 Burst 泡泡返回纯平移矩阵（sim 网格已在世界半径），对无 sim 的回退情况应用 `burstScale` 缩放。

4.13.2 鸿蒙端异步破裂仿真

在 Windows 端，破裂物理在主线程同步执行。鸿蒙端由于 CPU 资源有限且 UI 线程不能阻塞，将 DBSTT 破裂仿真放在独立的工作线程 (BurstSimWorkerThread) 中异步执行。主线程在每帧 `eglSwapBuffers` 之后调用 `RequestBubbleSimUpdates()`，将所有 Burst 状态泡泡的 ID 提交给工作线程。工作线程通过条件变量唤醒，对每个泡泡执行 `sim->update()` 并将结果顶点位置和法线存入 `BurstSimSnapshot` 快照。主线程在渲染时通过 `ApplyLatestBurstSnapshot()` 以 `try_lock` 方式尝试读取最新快照并上传至 GPU，若锁竞争失败则沿用上一帧的网格。这一设计使破裂物理不阻塞渲染帧率，同时通过版本号 (version) 避免重复应用同一快照。

4.14 渲染连续性与体积守恒

4.14.1 三层连续性策略

系统的视觉连续性分为三个层面：

状态连续：`contactActivation` 从近场连续增长；`candidate` 和 `bonded` 共用同一套接触参数；`bonded` 首帧继承预接触进度和入射速度；`contact pair` 不因单帧距离抖动而立即销毁。

几何连续：每个泡泡始终使用同一份持久球壳 `Model`，不切换到另一套 `assembly` 几何；接触圆和接触平面连续变化；球壳通过同一平面裁剪；共享膜边界使用同一接触圆。

渲染连续：`shell` 和共享膜保持稳定的 `thickness scale`；共享膜使用柔性 `alpha` 和较弱折射；接触 `frame` 进行低通滤波；透明泡泡按距离排序。

4.14.2 体积守恒

每个泡泡保存 `targetVolume` (由初始半径计算)，半径会缓慢向目标体积对应的半径恢复。局部表面压缩后，系统根据控制点平均径向位移进行小范围整体缩放补偿。这是实时近似，并非严格的封闭三角网格体积积分。后续可考虑使用 $V = \frac{1}{6} \sum \mathbf{x}_i \cdot (\mathbf{x}_j \times \mathbf{x}_k)$ 进行精确体积投影。

4.15 动态泡泡管理

系统支持运行时动态创建与删除泡泡：

- **创建：**通过界面“Add”模式生成新泡泡，带朝向中心的初速度和少量切向速度，随后自动进入 `broad phase` 和 `contact graph`。

- **删除**: 通过界面操作删除最近一个动态新增的泡泡, 同时清理该泡泡相关的 `contact pair` 和对应渲染 `model`。
- **身份稳定**: 泡泡身份使用稳定的 `BubbleId` (而非 `vector` 下标), `contact pair` 通过 `ResolveContactPairIndices()` 映射到当前数组下标, 确保删除或重排后模型不错位。

此外, 系统提供两套预置演示场景: 通过界面 **Demo** 按钮循环切换双泡专项演示 (等尺寸稳定双泡 o 小泡融合 o 短暂接触后分离), 通过 **Scene** 按钮重置综合开场场景 (中央三个真实模拟泡泡形成接触网络, 外围包含预设融合泡泡对和独立中型泡泡, 以及若干仅参与渲染的装饰小泡泡)。

4.16 当前局限

- 尚未形成统一的 **Plateau border**: `AB`、`AC`、`BC` 共享膜仍是独立 `contact patch`, 三张膜在中央可能互相穿过或留下小缝隙。
- 球壳边界与弯曲共享膜边界位置可对齐, 但各自法线独立计算, 透明折射中仍可能看到亮环或角度跳变。
- 膜厚仍为标量 (每个 `pair` 一个值), 无法表达重力排液、局部黑膜、风吹颜色流和接触区域材料交换。
- 透明多层渲染仍有限: 使用排序、`alpha` 混合和 `FBO` 折射。泡泡数量增加后可能出现绘制顺序错误和黑带。
- 球壳 `shader` 最多接收四个接触平面。真正多泡沫中一个泡泡可能有更多邻居。
- 融合概率模型是面向实时视觉的经验模型, 不是完整流体或膜排液求解器。

第 5 章 交互与演示

5.1 交互实现

5.1.1 触控交互（HarmonyOS 端）

在 HarmonyOS 设备上，系统以触控为主要交互方式，核心演示路径由三类手势构成：

表 5.1 触控交互与界面操作

操作	效果
Orbit 模式：单指拖动	旋转相机，围绕泡泡观察。
Orbit 模式：双击空白处	重置相机视角。
双指缩放	改变相机距离。
Add 模式：长按 + 拖动 + 松手	预览并放置新泡泡，自动进入接触仿真。
Pop 模式：点击泡泡	触发 DBSTT 物理破裂动画。
Scene 按钮	重置综合开场场景。
Demo 按钮	循环切换三种双泡演示（稳定双泡/融合/分离）。
Wind On/Off 按钮	开启或关闭全局风场。
设置面板	调整泡泡半径、生成深度、风场强度等参数。

在页面层，ArkTS 会先根据 XComponent 的实际区域将输入事件映射到局部坐标，再通过 NAPI 把坐标、事件类型和视口尺寸传给 Native 渲染层。单指点击主泡泡时，程序把触摸位置转换为归一化屏幕坐标 `g_TouchPoint`，并记录触摸强度和局部移动速度 `g_TouchVelocity`。当单指移动超过较小阈值后，ArkTS 侧会优先把后续移动解释为 `rotateCamera()` 输入，因此连续大幅拖动并非局部波纹的主要触发方式。

5.1.2 桌面端开发环境

Windows 桌面端（GLFW + OpenGL）用于算法开发、Shader 调试和效果验证。桌面端通过键鼠提供完整的参数调节接口（Fresnel 指数、折射强度、薄膜厚度、表面张力等 20 余项可调参数），但最终提交以 HarmonyOS 端为准。桌面端的核心交互逻辑（泡泡接触、融合、破裂、风场）已完整移植至 HarmonyOS Native 层。

5.1.3 界面参数

HarmonyOS 端将主要控制项整理为移动端界面按钮和参数面板。当前界面主要包含模式切换、仿真控制和参数滑块三类入口：

表 5.2 界面参数控制

界面项	参数	作用
模式按钮	Kim / Airy	切换两种主要演示模式；Native 层仍保留 Spectral LUT 便于内部对比。
状态按钮	暂停仿真 / 重置	暂停或继续主泡泡表面模拟，并同步冻结或恢复基于时间的视觉变化；重置时恢复默认参数。
光学参数	干涉膜厚、折射强度、环境反射	调整彩色条纹尺度、背景扭曲强度和環境反射权重。
仿真参数	厚度扰动、表面张力、边缘畸变	调整动态膜厚幅度、主泡泡恢复趋势和轮廓额外折射增强。
视图参数	相机距离	调整观察远近，便于展示单泡泡细节或多泡泡整体构图。
状态显示	FPS、当前模式、交互提示	方便现场展示系统帧率、当前材质模式和操作说明。

除 ArkTS 界面当前开放的滑块外，Native 接口层还保留了 `uFresnelPower` 和 `uMaxOffsetRatio` 对应的调节入口，用于内部调参与折射稳定性验证；它们没有直接暴露在比赛演示面板中。

5.2 演示场景

5.2.1 综合开场场景（Scene 按钮）

程序启动后默认进入综合开场场景：中央三个真实模拟泡泡（半径约 0.48、0.44、0.40）从松散三角形位置向更靠近的 home position 移动，自然形成多条接触关系；右侧一大一小两个真实泡泡预设为融合结果；左侧保留一个独立中型泡泡；四周分布若干仅参与渲染的装饰小泡泡（只做低频漂动，不进入 broad phase、接触图或融合计算）。

5.2.2 双泡专项演示（Demo 按钮）

按 Demo 按钮清空装饰泡泡并循环切换三种双泡演示：

1. **等尺寸稳定双泡**：两泡泡半径相同，形成 8 字形轮廓，共享膜接近平面，接触圆有轻微周期波动。
2. **小泡融合**：小泡泡尺寸约为大泡泡的 60%，经过共享膜变薄、破裂、颈部扩张后，小泡泡被大泡泡吸收。融合后按体积守恒确定新半径。
3. **短暂接触后分离**：两泡泡短暂接触形成共享膜后，接触膜收缩，粘结合约释放，双方沿接触法线分离并恢复独立运动。

演示场景使用固定结果，便于观察视觉过程；用户动态生成的普通泡泡仍使用概率结果选择。

5.3 风场演示

全局风场默认关闭。通过 **Wind** 按钮开启后，方向性风力作用于所有泡泡。小泡泡具有更高的风力响应（更易被吹动），大泡泡运动相对稳定。关闭风后，已有速度通过阻尼平滑衰减，泡泡逐渐回到有界摆动范围。可通过设置面板调整风力强度（0.0–0.45）和风向。

第 6 章 结果与分析

6.1 系统成果

当前决赛版本在初赛工作的基础上，实现了一个面向比赛提交和现场演示的完整透明泡泡渲染与交互系统。主要成果包括：

6.1.1 渲染与光学

- 通过多阶段 FBO 近似双界面折射，背景参照物和天空盒能在泡泡中产生明显扭曲。
- 三种薄膜虹彩模式已在 Native 层实现：Kim2012 实时三波长公式计算、Spectral LUT 预计算纹理查表、Belcour Airy 多波长 Airy 反射近似。界面开放 Kim 与 Airy 两种展示模式，便于说明实时公式与多波长近似之间的差异。
- 动态膜厚让虹彩随时间缓慢流动，不依赖静态纹理。程序化膜厚由多层 Simplex 噪声、排液趋势项和触摸扰动共同生成，使用世界法线方向（而非 UV 坐标）作为噪声输入，避免球体 UV 接缝处的明显断裂。

6.1.2 多泡泡交互

- 基于空间哈希和接触图的多泡泡仿真框架：支持运行时动态创建与删除泡泡，泡泡数量可变（默认上限 8，硬上限 16）。
- 接触激活函数实现从接近、预接触、成膜到结果锁定的平滑过渡，避免单帧状态切换。
- 接触圆弹簧-阻尼动力学与多接触位置约束迭代（每帧 6 次），支持一个泡泡同时参与多条接触关系。
- 每泡 26 个持久表面控制点，弹簧-阻尼更新，支持多接触变形聚合、接触帽压缩和接触环鼓包。
- Young-Laplace 弯曲共享膜：等尺寸泡泡间为平面膜，不等尺寸时膜向大泡（低压）一侧弯曲，使用解析球冠表示且边界始终与原接触平面对齐。
- 三种接触结果：分离（粘结合约释放后柔性分离）、稳定 8 字双泡（独立腔体，共享膜保留）、融合（小泡被大泡吸收，体积守恒）。
- 泡泡身份使用稳定 BubbleId，支持运行时增删而不导致模型错位。

6.1.3 动力学与交互

- 方向性风场：可调节强度与方向，小泡泡响应更强，关闭后速度阻尼平滑衰减。

- 每泡 DBSTT 涡流片仿真：泡泡破裂时激活，膜回缩由涡流片物理模型驱动而非预设动画。
- 两套预置演示场景：综合开场场景（Scene 按钮）和双泡专项循环演示（Demo 按钮），覆盖稳定双泡、融合、分离三种典型交互。
- 完整 HarmonyOS 端触控与界面交互，支持动态生成、删除、参数调节和场景切换。

6.1.4 工程完整性

- HarmonyOS 工程（ArkTS 页面 + NAPI 接口 + Native 渲染层）和 Windows 桌面工程（GLFW + OpenGL）已形成完整的双平台开发与提交链路。
- 可执行 HAP 程序已整理入提交目录。

6.2 效果展示

6.2.1 多泡泡接触场景



图 6.1 多泡泡接触场景：三个真实模拟泡泡形成接触网络，右侧为融合结果，左侧为独立泡泡

6.2.2 共享膜与接触圆



图 6.2 接触圆与共享膜：泡泡间形成圆形共享膜，球壳在接触平面处被裁剪

6.2.3 三种接触结果对比

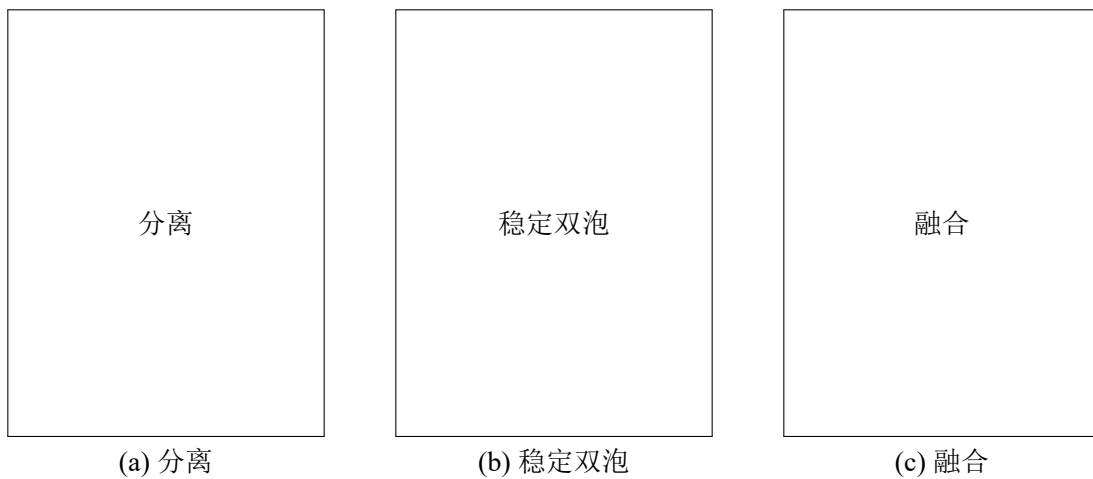


图 6.3 三种接触结果对比示意图（请替换为实际截图）

6.2.4 弯曲共享膜

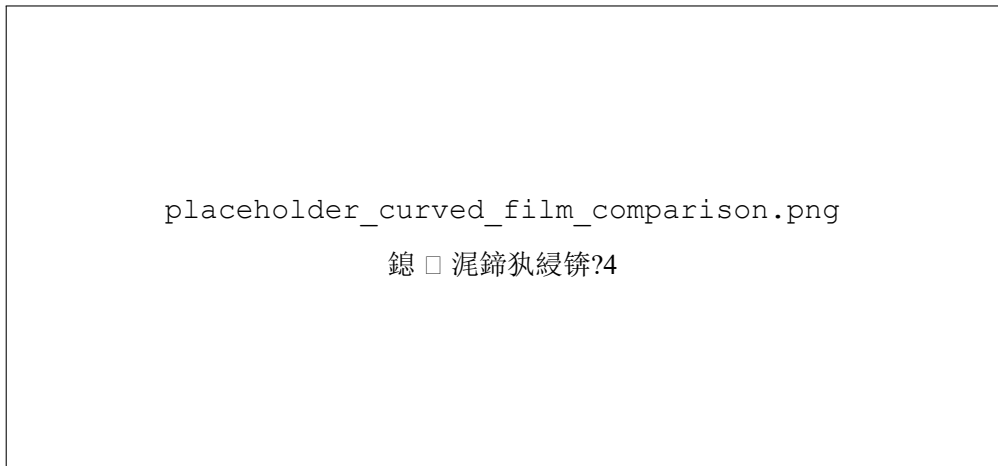


图 6.4 不同尺寸比下的弯曲共享膜：左为等尺寸（平面膜），中为大泡-小泡（膜向大泡凸出），右为极端尺寸比

6.3 性能分析

系统在 Windows 桌面端（Release 构建）和 HarmonyOS 设备上均能稳定运行。由于桌面端为开发调试环境，此处以 HarmonyOS 端性能为主要参考。

表 6.1 性能参考数据

场景	泡泡数	帧率 (FPS)
单泡泡 (DBSTT 仿真)	1	约 60
综合开场场景	8+	约 30
双泡泡接触交互演示	2	约 50

以上性能数据在华为 Mate 70 Pro 真机上测得。主要性能开销来源：

- 多阶段 FBO 合成（每帧至少 5 个 pass）；
- 接触图位置约束迭代（ $O(K \cdot I)$ ，其中 K 为接触边数， $I = 6$ 为迭代次数）；
- 每泡表面控制点更新（ $O(N \cdot C)$ ， N 为泡泡数， $C = 26$ 为控制点数）；
- DBSTT 涡流片仿真的 Biot-Savart 速度求解（ $O(V \cdot T)$ ， V 为顶点数， T 为三角形数）。

6.4 局限与展望

当前系统存在以下明确局限：

- **屏幕空间折射的固有局限**：不能表现屏幕外物体，也不能严格模拟多泡泡之间的相互折射。
- **Plateau border 缺失**：多接触相交处尚未形成符合表面张力平衡的 120 度交汇结构，共享膜在中央可能互相穿过或留下缝隙。

- **膜厚为标量**: 每个接触对只有一个膜厚值, 无法表达重力排液、局部黑膜和接触区域材料交换。
- **透明排序**: 泡泡数量增加后, 固定距离排序 +alpha 混合可能出现顺序错误和黑带。
- **Shader 接触平面数量限制**: 球壳 shader 最多接收四个接触平面, 限制了一个泡泡的最大邻居数。
- **融合概率模型**: 当前基于尺寸比、接触时间等的经验概率模型, 不是完整膜排液或流体求解器。
- **DBSTT 简化**: 仿真仅用于单闭合泡泡, 不支持原论文中的泡沫拓扑变化和 Plateau 边界。
- **Belcour Airy 简化**: 是实时 shader 中的项目级简化, 不包含完整微表面模型和解析光谱预积分。

后续迭代的优先方向:

1. 实现多泡泡之间的相互折射和反射, 提升多泡泡场景的光学真实感。
2. 建立显式的 ContactPatch 边界环和 PlateauBorder, 使多接触交汇处符合物理规律。
3. 实现低分辨率二维膜厚场, 支持排液、颜色流和接触区材料交换。
4. 采用 weighted blended OIT 或两层 depth peeling 改善透明排序。
5. 将接触数据从 uniform 数组扩展至 texture buffer 或 SSBO, 支持任意数量的邻居。
6. 实现严格的封闭三角网格体积积分与投影, 支持更深挤压和动态拓扑变化。

6.5 总结

本项目从透明肥皂泡的主要视觉现象出发, 实现了一个面向赛题二决赛提交的 HarmonyOS 渲染系统。渲染方面, 项目通过多阶段 FBO 合成近似双界面折射, 实现了 Kim2012、Spectral LUT 和 Belcour Airy 三种薄膜虹彩模式。动力学方面, 项目构建了以接触图为核心的多泡泡交互框架: 引入空间哈希加速、连续接触激活、接触圆弹簧-阻尼动力学、持久表面控制点变形、Young-Laplace 弯曲共享膜, 以及分离/稳定双泡/融合三种接触结果。辅助系统包括方向性风场、每泡 DBSTT 涡流片仿真、动态泡泡增删和两套预置演示场景。

系统在 HarmonyOS 和 Windows 双平台均形成完整的开发与演示链路。决赛版本将初赛的单泡泡展示系统扩展为真正意义上的多泡泡交互仿真平台, 使最终结果更符合比赛对“效果、交互、完整性”的要求。

参 考 文 献

- [1] WYMAN C. An approximate image-space approach for interactive refraction[C]//ACM SIG-GRAPH 2005 Sketches. ACM, 2005: 1050-1053.
- [2] KIM N, PARK K. Real time rendering iridescent colors appearing on soap bubbles[M/OL]//Lecture Notes in Electrical Engineering: volume 181 Embedded and Multimedia Computing Technology and Service. Springer, 2012: 377-384. DOI: 10.1007/978-94-007-5075-3_45.
- [3] BELCOUR L, BARLA P. A practical extension to microfacet theory for the modeling of varying iridescence[J/OL]. ACM Transactions on Graphics, 2017, 36(4): 65:1-65:14. DOI: 10.1145/3072959.3073620.
- [4] ISHIDA S, SYNAK P, NARITA F, et al. A model for soap film dynamics with evolving thickness[J/OL]. ACM Transactions on Graphics, 2020, 39(4): 31:1-31:11. DOI: 10.1145/3386569.3392405.
- [5] DA F, BATTY C, WOJTAN C, et al. Double bubbles sans toil and trouble: Discrete circulation-preserving vortex sheets for soap films and foams[J/OL]. ACM Transactions on Graphics, 2015, 34(4): 149:1-149:9. DOI: 10.1145/2767003.
- [6] BUSARYEV O, DEY T K, WANG H, et al. Animating bubble interactions in a liquid foam [J/OL]. ACM Transactions on Graphics, 2012, 31(4): 63:1-63:8. DOI: 10.1145/2185520.2185559.
- [7] DENG Y, WANG M, KONG X, et al. A moving eulerian-lagrangian particle method for thin film and foam simulation[J/OL]. ACM Transactions on Graphics, 2022, 41(4): 75:1-75:14. DOI: 10.1145/3528223.3530173.
- [8] SAYE R I, SETHIAN J A. Multiscale modelling of evolving foams[J/OL]. Journal of Computational Physics, 2016, 315: 273-301. DOI: 10.1016/j.jcp.2016.02.064.
- [9] KIM B, LIU Y, LLAMAS I, et al. Simulation of bubbles in foam with the volume control method[J/OL]. ACM Transactions on Graphics, 2007, 26(3): 98:1-98:10. DOI: 10.1145/1276377.1276503.